EE216 Reconfigurable Computing

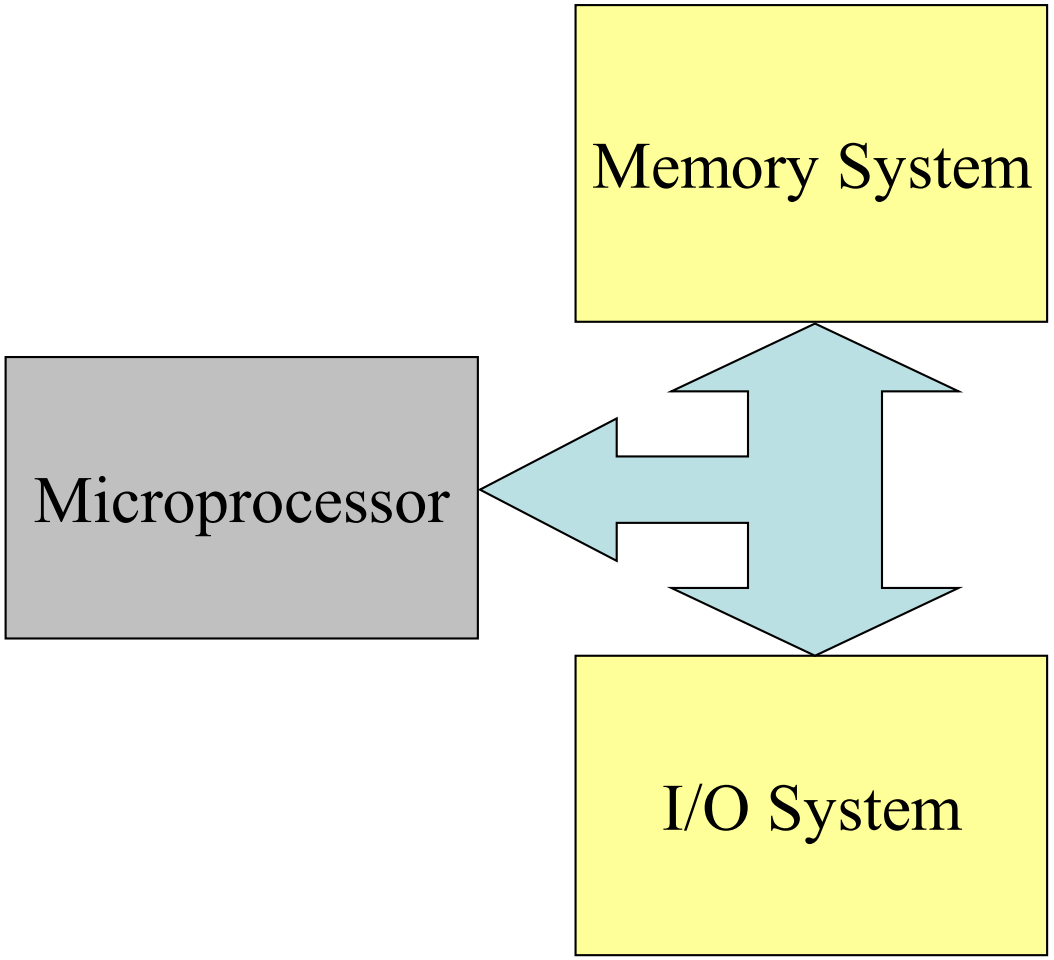
Topic 8：Processor and Accelerator Based Computing Architectures

Prof. Yajun Ha

ShanghaiTech University
Shanghai, China

上海科技大学
ShanghaiTech University

General Block Diagram of Microprocessor Systems

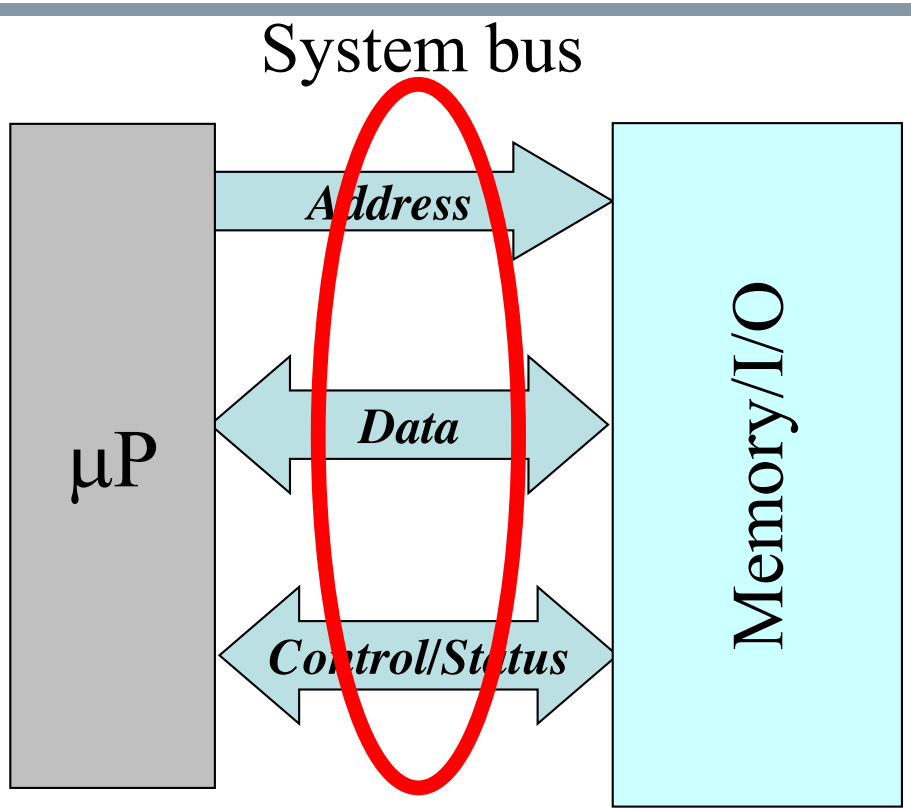


- **Memory system stores the instructions and data**
- **Microprocessor gets instructions and data from memory, executes instructions on data, and stores the processed data back to memory.**
- **I/O system extends the memory system to provide peripheral specific data to be processed by microprocessor.**
- **Our module studies three key words: Architecture, Programming and Interfacing**

Processor & Accelerator based Computing Architectures

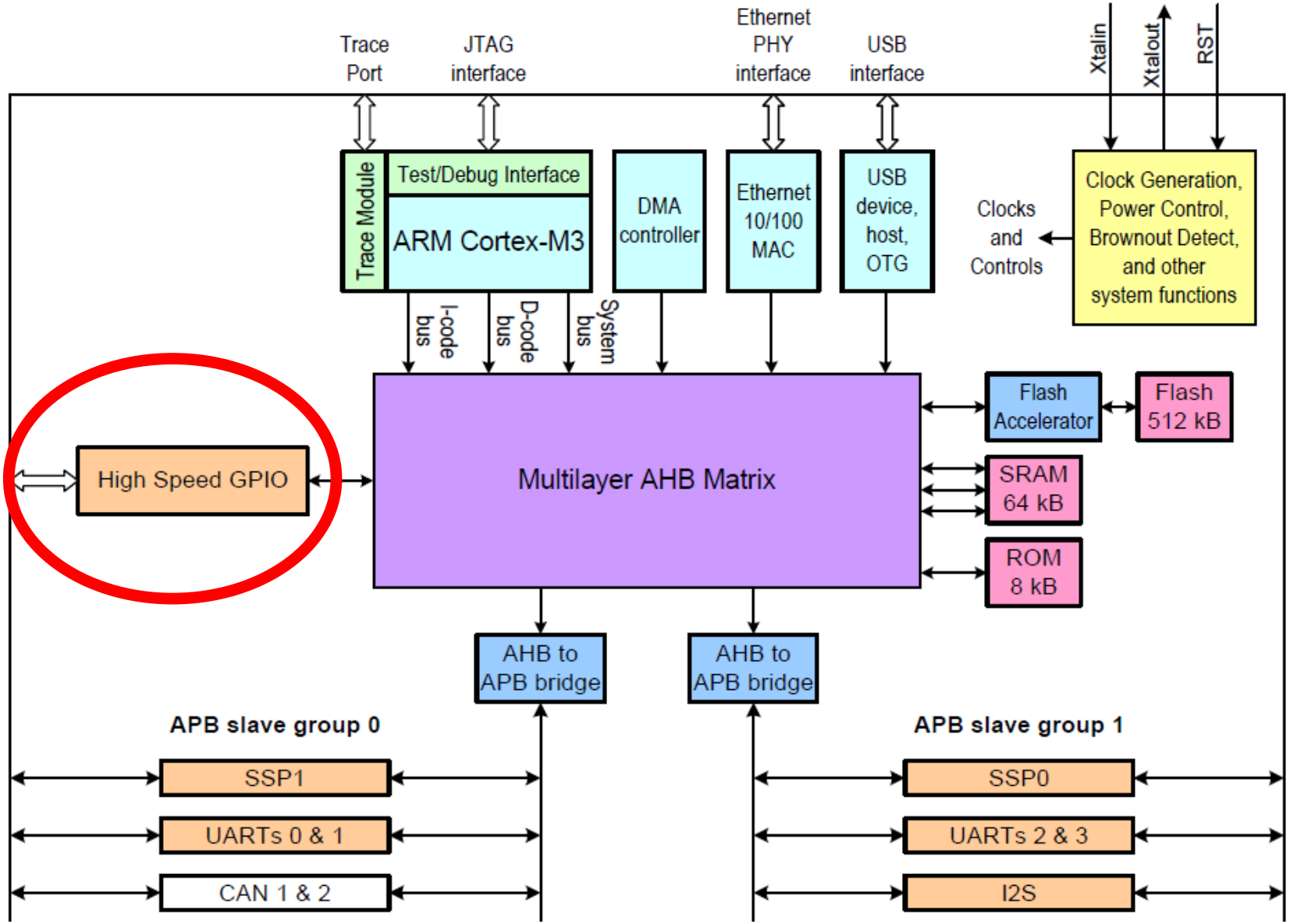
- ☛ I/O interfacing
- Addressing
- Interrupt
- Direct memory access

A First Look at Memory and I/O Interfacing

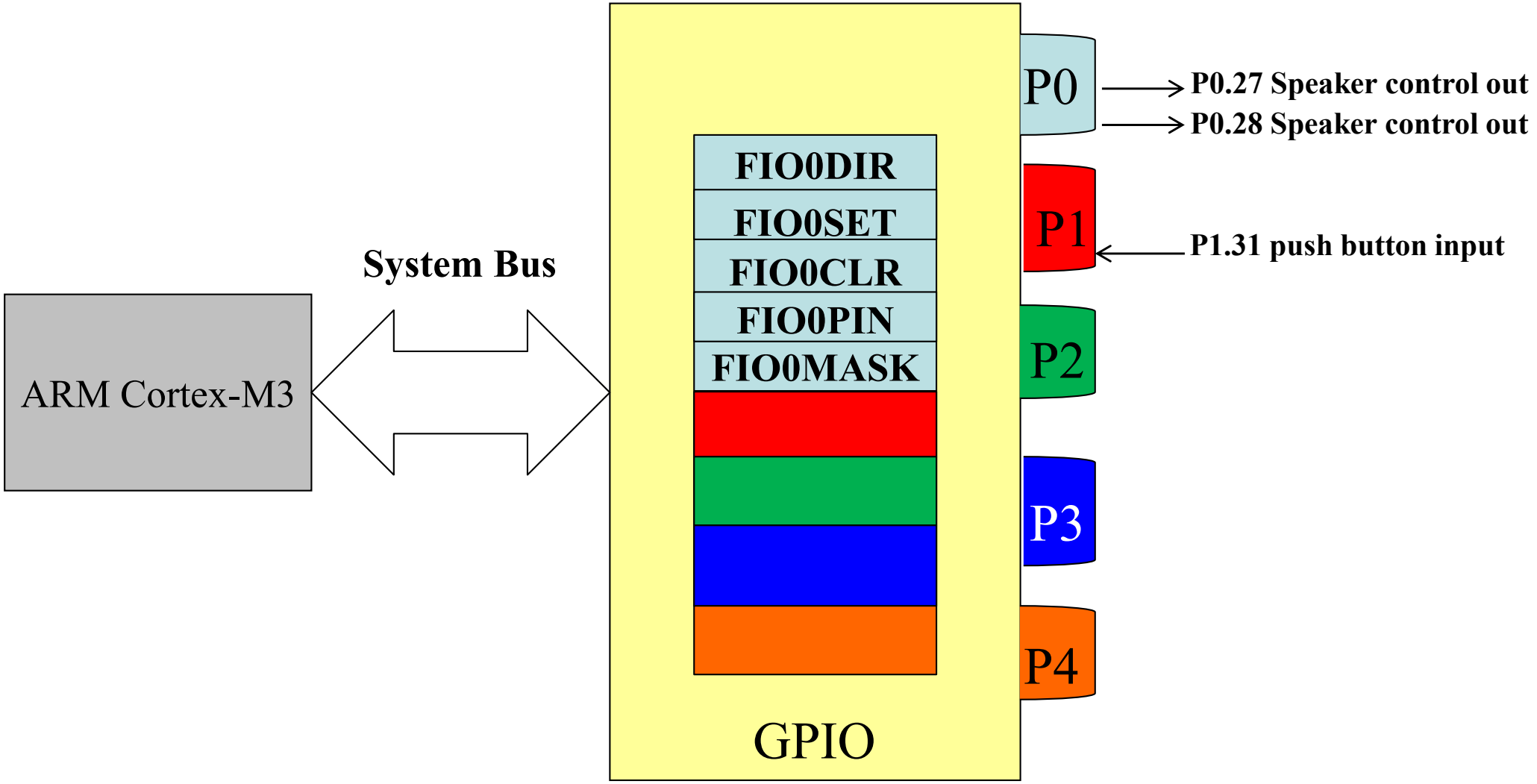


- μP is interfaced to memory (I/O) to read inputs and write results.
- Address bus decides which range of memory (I/O) space to be accessed by μP.
- Control bus decides READ or WRITE operation of the chosen memory (I/O) locations.
- Data Bus provides the data transfer channel between memory (I/O) and μP.

NXP LPC17xx Simplified Block Diagram



GPIO Simplified Block Diagram



Pins in P0 can be configured by FIO0DIR register as input or output pins.
Pins in P* can be configured by FIO*DIR registers as input or output pins.

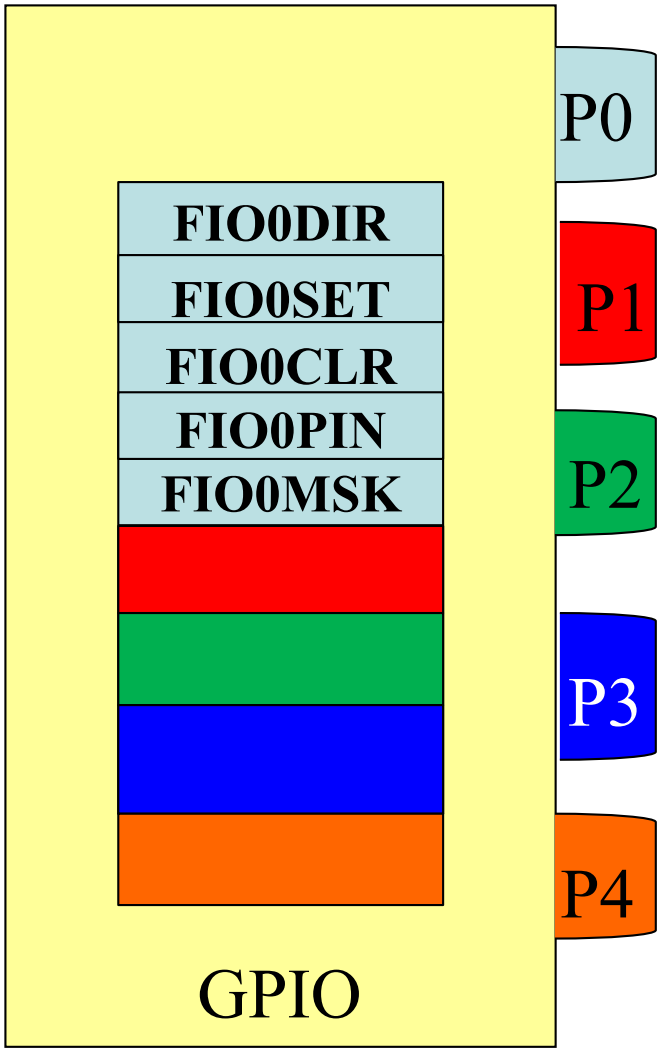
Overview of GPIO (General Purpose Input/Output)

- GPIO is a generic pin on a chip
- GPIO behavior (including whether it is an input or output pin) can be controlled (programmed) through software.
- GPIO pins have no special purpose themselves, and go unused by default.
- The idea for GPIO is that:
  - Chips might need additional digital control lines in the future
  - Designers have to arrange additional circuitry to provide them
  - Having GPIOs available from the chip can save this hassle

- *Note*: LPC1769 has about 70 GPIO lines, organized into ports and pins

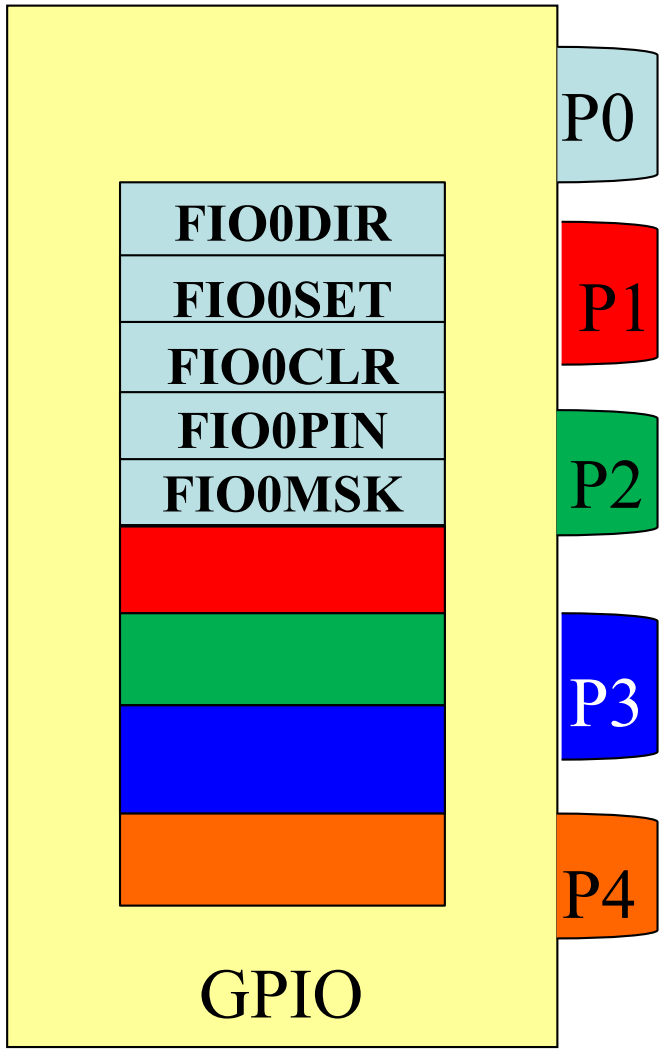
GPIO Capabilities

- Direction: GPIO pins can be configured to be input or output by FIO*DIR
- Enable mask (GPIO mask): GPIO pins can be enabled/disabled by FIO*MASK
- Input values are readable from FIO*PIN (typically high=1, low=0)
- Output values are writable/readable from FIO*PIN
- SET/CLR pins: values in GPIO pins can be set/cleared by FIO*SET/FIO*CLR register



GPIO Registers for Digital IO Functions

- Fast GPIO Port Direction Registers: FIOxDIR (x = 0 to 4)
- Fast GPIO Port Output Set Register: FIOxSET (x = 0 to 4)
- Fast GPIO Port Output Clear Registers: FIOxCLR (x = 0 to 4)
- Fast GPIO Port Pin Value Registers: FIOxPIN (x = 0 to 4)
- Fast GPIO Port Mask Registers: FIOxMASK (x = 0 to 4)



GPIO Register Map

- Need to know the register address to access and configure the GPIO registers.

Example:

Configure all pins in P1 as inputs:

```
MOV R0, #0
STR R0, FIO1DIR
```

Generic Name	Access	Reset value[1]	PORTn Register Name & Address
FIODIR	R/W	0	FIO0DIR - 0x2009 C000 FIO1DIR - 0x2009 C020 FIO2DIR - 0x2009 C040 FIO3DIR - 0x2009 C060 FIO4DIR - 0x2009 C080
FIOMASK	R/W	0	FIO0MASK - 0x2009 C010 FIO1MASK - 0x2009 C030 FIO2MASK - 0x2009 C050 FIO3MASK - 0x2009 C070 FIO4MASK - 0x2009 C090
FIOPIN	R/W	0	FIO0PIN - 0x2009 C014 FIO1PIN - 0x2009 C034 FIO2PIN - 0x2009 C054 FIO3PIN - 0x2009 C074 FIO4PIN - 0x2009 C094
FIOSET	R/W	0	FIO0SET - 0x2009 C018 FIO1SET - 0x2009 C038 FIO2SET - 0x2009 C058 FIO3SET - 0x2009 C078 FIO4SET - 0x2009 C098
FIOCLR	WO	0	FIO0CLR - 0x2009 C01C FIO1CLR - 0x2009 C03C FIO2CLR - 0x2009 C05C FIO3CLR - 0x2009 C07C FIO4CLR - 0x2009 C09C

LPC_GPIO0 to LPC_GPIO4

```
//LPC17xx.h
/*********************************************************************/
/*                        Peripheral memory map                      */
/*********************************************************************/
/* Base addresses                                                    */
#define LPC_FLASH_BASE        (0x00000000UL)
#define LPC_RAM_BASE          (0x10000000UL)
#define LPC_GPIO_BASE         (0x2009C000UL)
#define LPC_APB0_BASE         (0x40000000UL)
#define LPC_GPIOINT_BASE      (LPC_APB0_BASE + 0x28080)
/* GPIOs                                                             */
#define LPC_GPIO0_BASE        (LPC_GPIO_BASE + 0x00000)
#define LPC_GPIO1_BASE        (LPC_GPIO_BASE + 0x00020)
#define LPC_GPIO2_BASE        (LPC_GPIO_BASE + 0x00040)
#define LPC_GPIO3_BASE        (LPC_GPIO_BASE + 0x00060)
#define LPC_GPIO4_BASE        (LPC_GPIO_BASE + 0x00080)
/*********************************************************************/
/*                        Peripheral declaration                     */
/*********************************************************************/
#define LPC_SC                ((LPC_SC_TypeDef        *) LPC_SC_BASE       )
#define LPC_GPIO0             ((LPC_GPIO_TypeDef      *) LPC_GPIO0_BASE    )
#define LPC_GPIO1             ((LPC_GPIO_TypeDef      *) LPC_GPIO1_BASE    )
#define LPC_GPIO2             ((LPC_GPIO_TypeDef      *) LPC_GPIO2_BASE    )
#define LPC_GPIO3             ((LPC_GPIO_TypeDef      *) LPC_GPIO3_BASE    )
#define LPC_GPIO4             ((LPC_GPIO_TypeDef      *) LPC_GPIO4_BASE    )
#define LPC_GPIOINT           ((LPC_GPIOINT_TypeDef   *) LPC_GPIOINT_BASE  )
//…
```

GPIO Functions in MCU library

```
/* GPIO style ------------------------------ */
void GPIO_SetDir(uint8_t portNum, uint32_t bitValue, uint8_t dir);
void GPIO_SetValue(uint8_t portNum, uint32_t bitValue);
void GPIO_ClearValue(uint8_t portNum, uint32_t bitValue);
uint32_t GPIO_ReadValue(uint8_t portNum);
/* FIO (word-accessible) style ------------------------------ */
void FIO_SetDir(uint8_t portNum, uint32_t bitValue, uint8_t dir);
void FIO_SetValue(uint8_t portNum, uint32_t bitValue);
void FIO_ClearValue(uint8_t portNum, uint32_t bitValue);
uint32_t FIO_ReadValue(uint8_t portNum);
void FIO_SetMask(uint8_t portNum, uint32_t bitValue, uint8_t maskValue);
/* FIO (halfword-accessible) style ------------------------------ */
void FIO_HalfWordSetDir(uint8_t portNum, uint8_t halfwordNum, uint16_t bitValue, uint8_t
dir);
void FIO_HalfWordSetMask(uint8_t portNum, uint8_t halfwordNum, uint16_t bitValue, uint8_t
maskValue);
void FIO_HalfWordSetValue(uint8_t portNum, uint8_t halfwordNum, uint16_t bitValue);
void FIO_HalfWordClearValue(uint8_t portNum, uint8_t halfwordNum, uint16_t bitValue);
uint16_t FIO_HalfWordReadValue(uint8_t portNum, uint8_t halfwordNum);
/* FIO (byte-accessible) style ------------------------------ */
void FIO_ByteSetDir(uint8_t portNum, uint8_t byteNum, uint8_t bitValue, uint8_t dir);
void FIO_ByteSetMask(uint8_t portNum, uint8_t byteNum, uint8_t bitValue, uint8_t maskValue);
void FIO_ByteSetValue(uint8_t portNum, uint8_t byteNum, uint8_t bitValue);
void FIO_ByteClearValue(uint8_t portNum, uint8_t byteNum, uint8_t bitValue);
uint8_t FIO_ByteReadValue(uint8_t portNum, uint8_t byteNum);
```

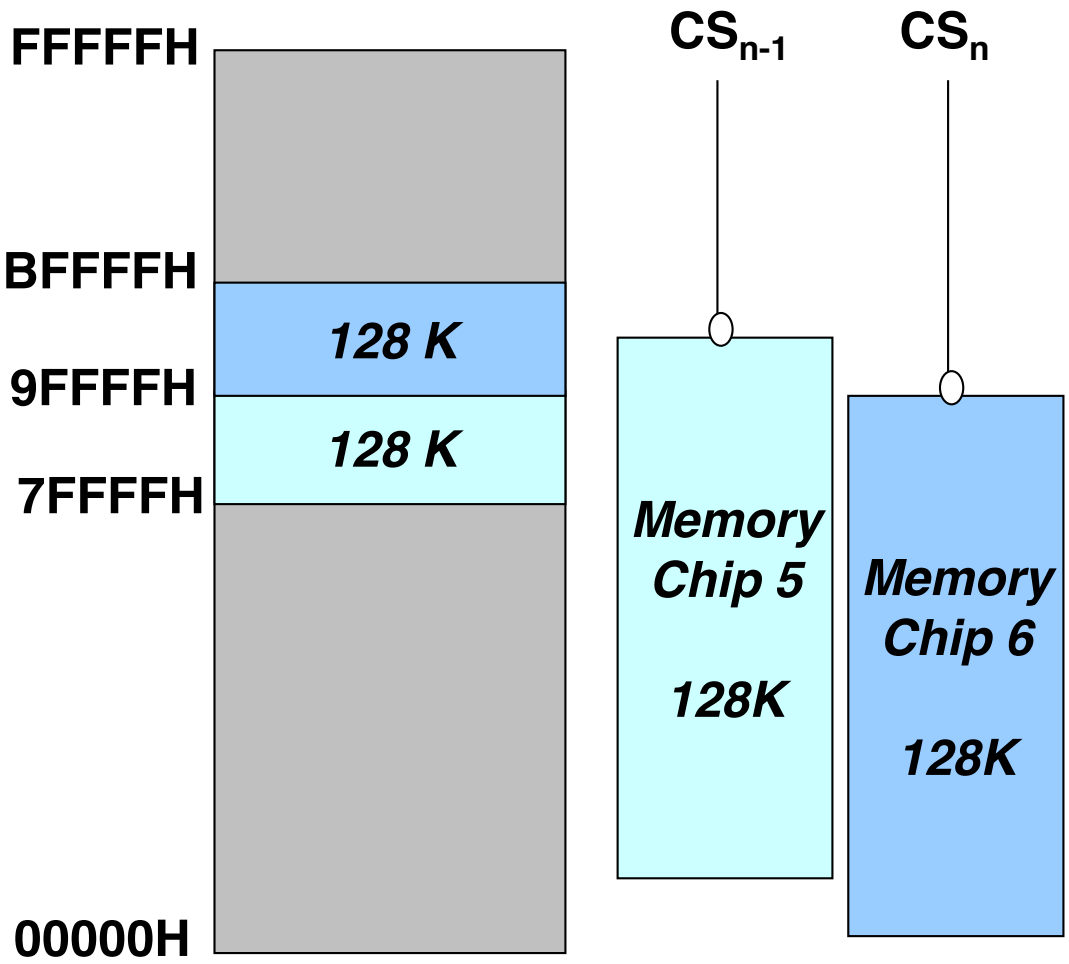
Processor & Accelerator based Computing Architectures

- I/O interfacing
- ☛ Addressing
- Interrupt
- Direct memory access

Memory Address Space vs Actual Memory Size

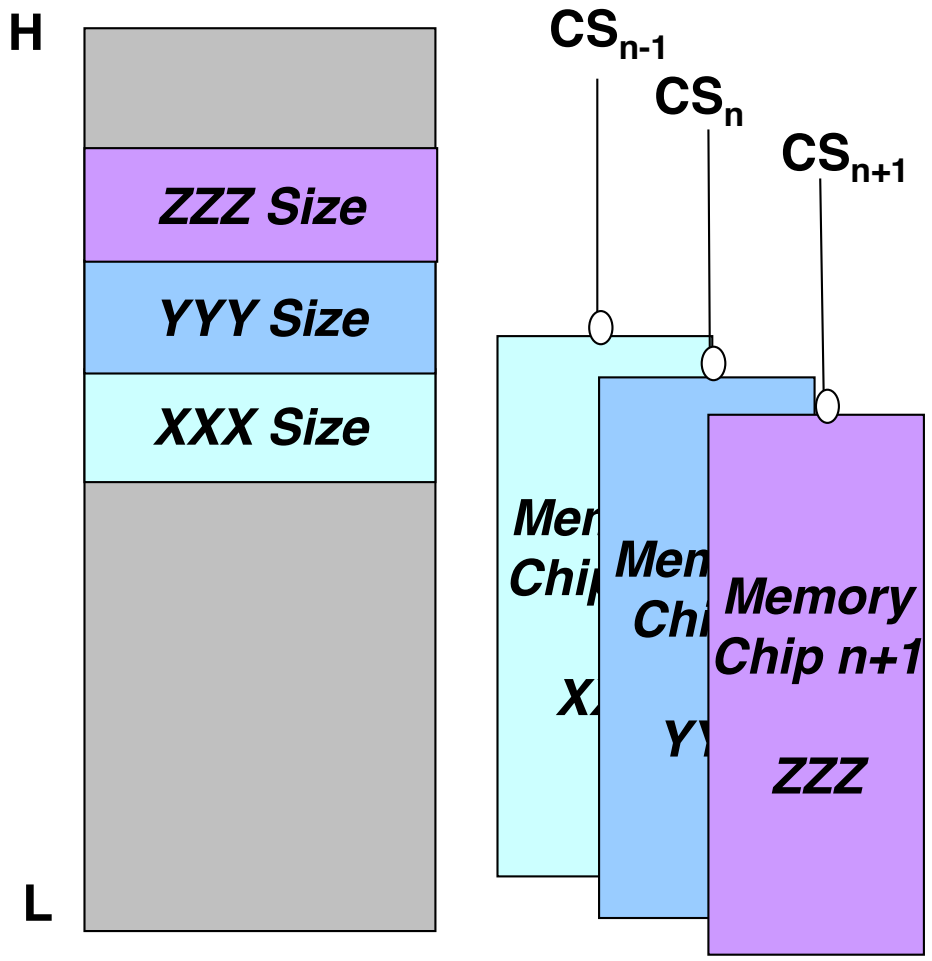
The memory address space is 2^n, where n is the number of address bus lines. In 8088/8086, n =20, thus its memory address space is 00000H-FFFFFH.

The actual memory size depends on how many memory locations have been covered by memory chips in the memory address space. In this example, the actual memory size is 256K.



How to Increase Memory Size of Your Computer

Although the memory address space of a computer has been fixed by its address bus size, buying and installing as many as possible more memory chips will increase the actual memory size of your computer.



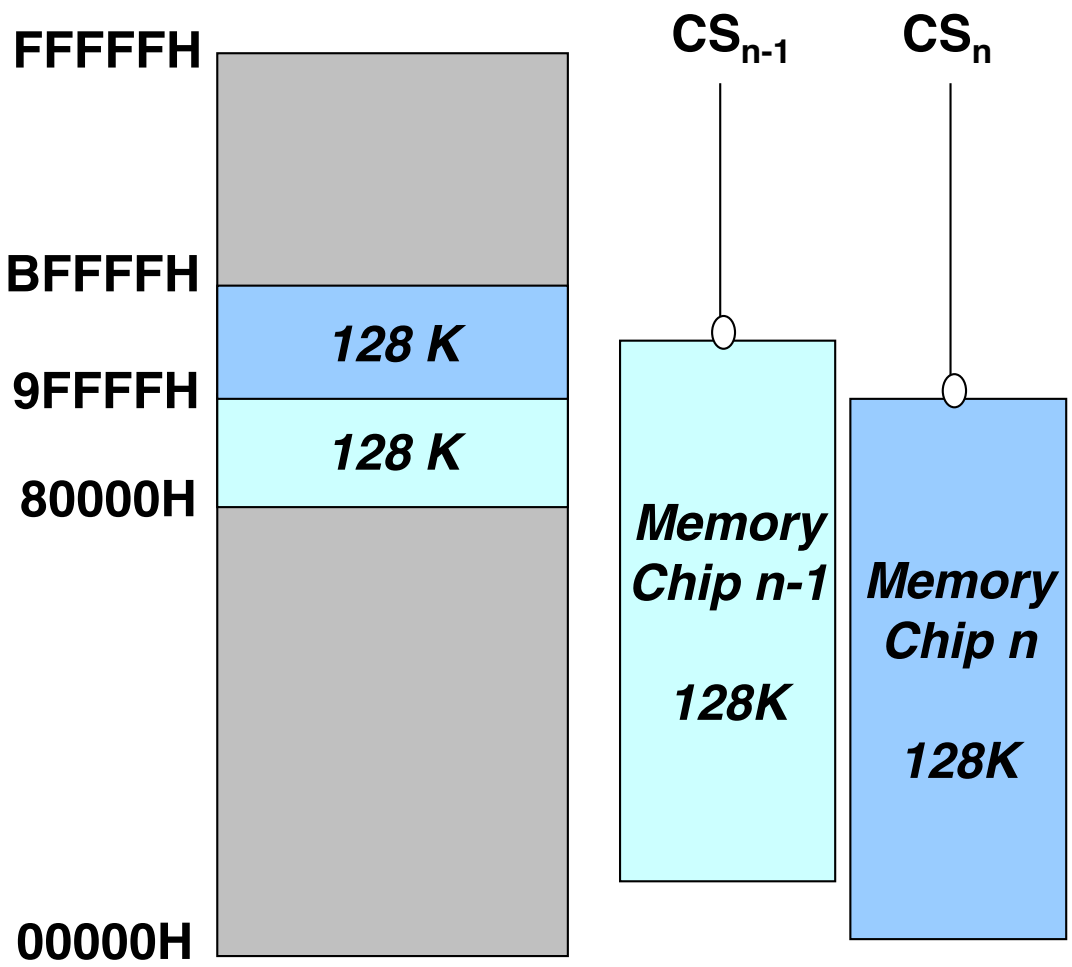
Memory Addressing Issues

1. Decide the **memory address map**. It shows which type of memory chip is mapped to which address range.

   Chip N-1: 80000H-9FFFFH
   Chip N: A0000H-BFFFFH

2. Derive the **logic function of chip select signal CS_n** for each memory chip based on the memory address map.

   $CS_{n-1} = A_{19}A_{18}\backslash A_{17}\backslash$
   $CS_n = A_{19}A_{18}\backslash A_{17}$



Memory Address Mapping

Memory address mapping decides:

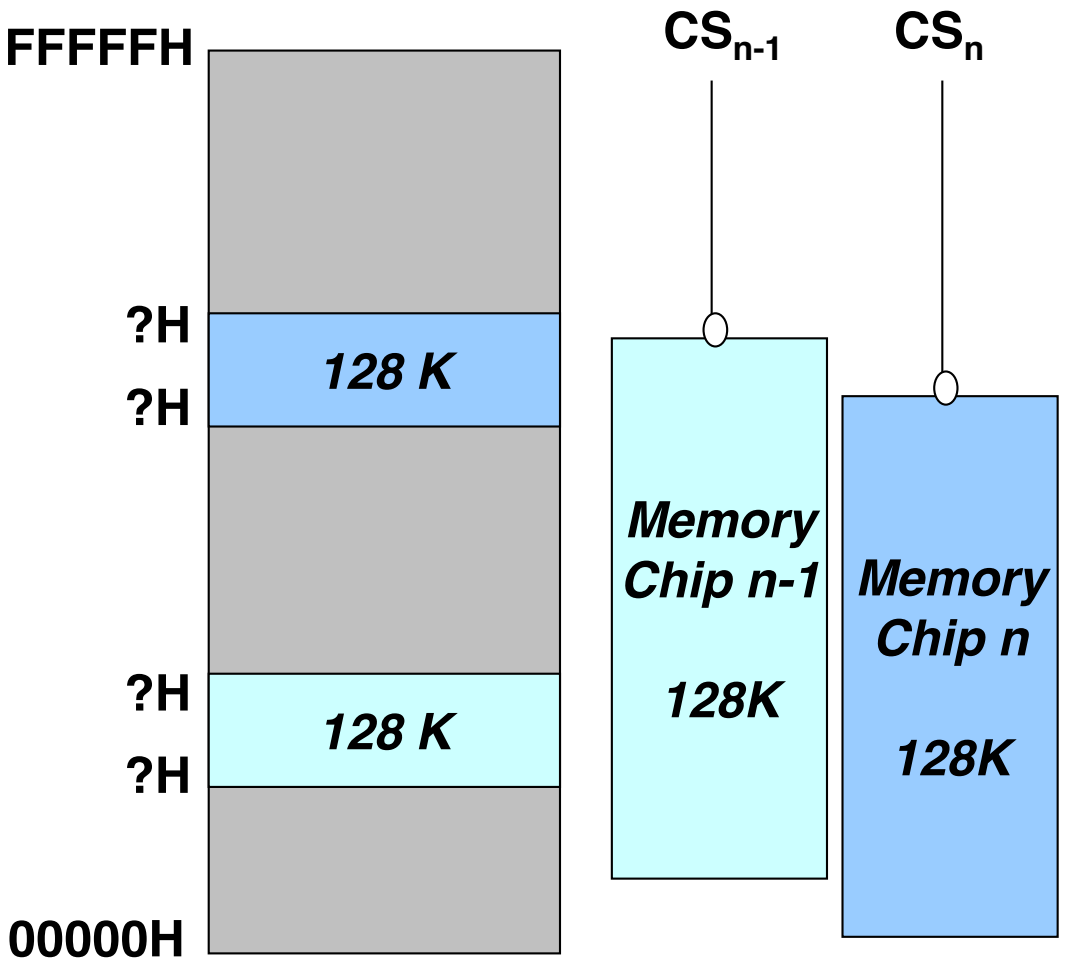
1. which type of memory chip (RAM or ROM)

2. should be mapped to which address range?

Address Mapping Results:

Chip N-1: Type, Range

Chip N: Type, Range



ROM vs RAM

• ROM stands for read-only memory. A ROM chip will maintain its information even when power goes off.

•RAM stands for random-access memory. RAM contains bytes of information, and the microprocessor can read or write to those bytes depending on whether the RD or WR line is signaled. One problem with today's RAM chips is that they forget everything once the power goes off. That is why the computer needs ROM.

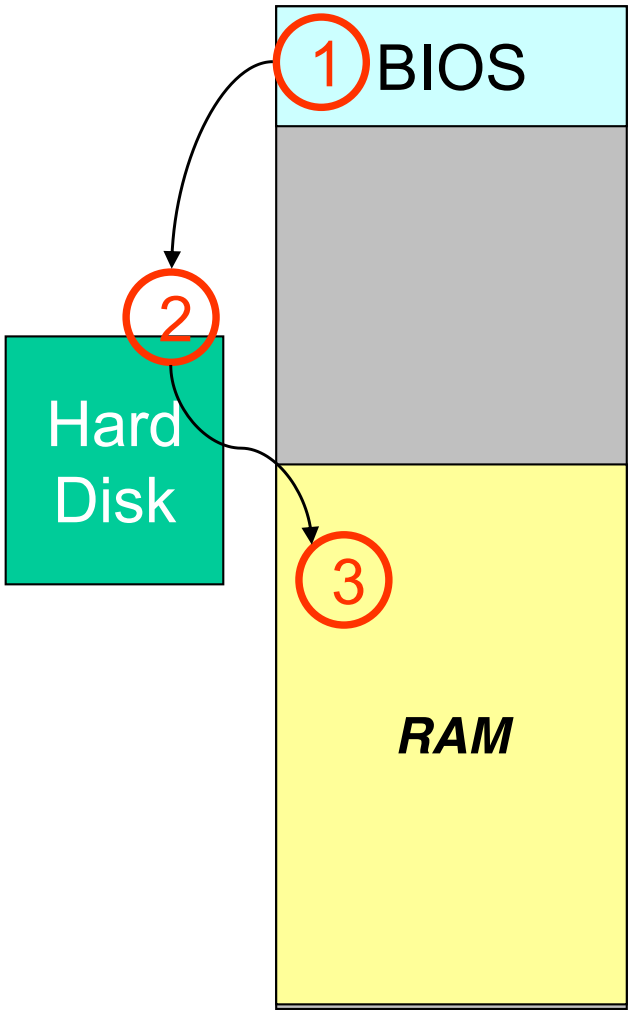
How a Computer Loads Data When It Starts (Booting)

Nearly all computers contain some amount of ROM and RAM. On a PC, the ROM is called the *BIOS* (Basic Input/Output System).

1. When the microprocessor starts, it begins executing instructions it finds in the BIOS. The BIOS instructions do things such as testing the hardware in the machine.

2. Then it goes to the hard disk to fetch the boot sector. This boot sector is another small program, and the BIOS stores it in RAM after reading it off the disk.

3. The microprocessor then begins executing the boot sector's instructions from RAM. The boot sector program will tell the microprocessor to fetch something else from the hard disk into RAM, which the microprocessor then executes, and so on.



Two Constraints in Memory Address Map

- **Where is the reset vector of the microprocessors**
  - in case of the Intel 8086: CS:IP = FFFF0h

    Reset vector stores the first instruction to be executed in BIOS.
  - Requires at least one EPROM/ROM chip to be mapped at the highest addresses

- **Where is the interrupt vector table stored?**
  - in case of the Intel 8086: at 00000H-003FFh

    The first 1K locations of memory address space
  - Requires at least one RAM chip to be mapped at the lowest addresses

- **For the simplest system, requires**
  - 1 EPROM chip from FFFFFh downward
  - 1 RAM chip from 00000h upward

Derive the Chip Select Signal

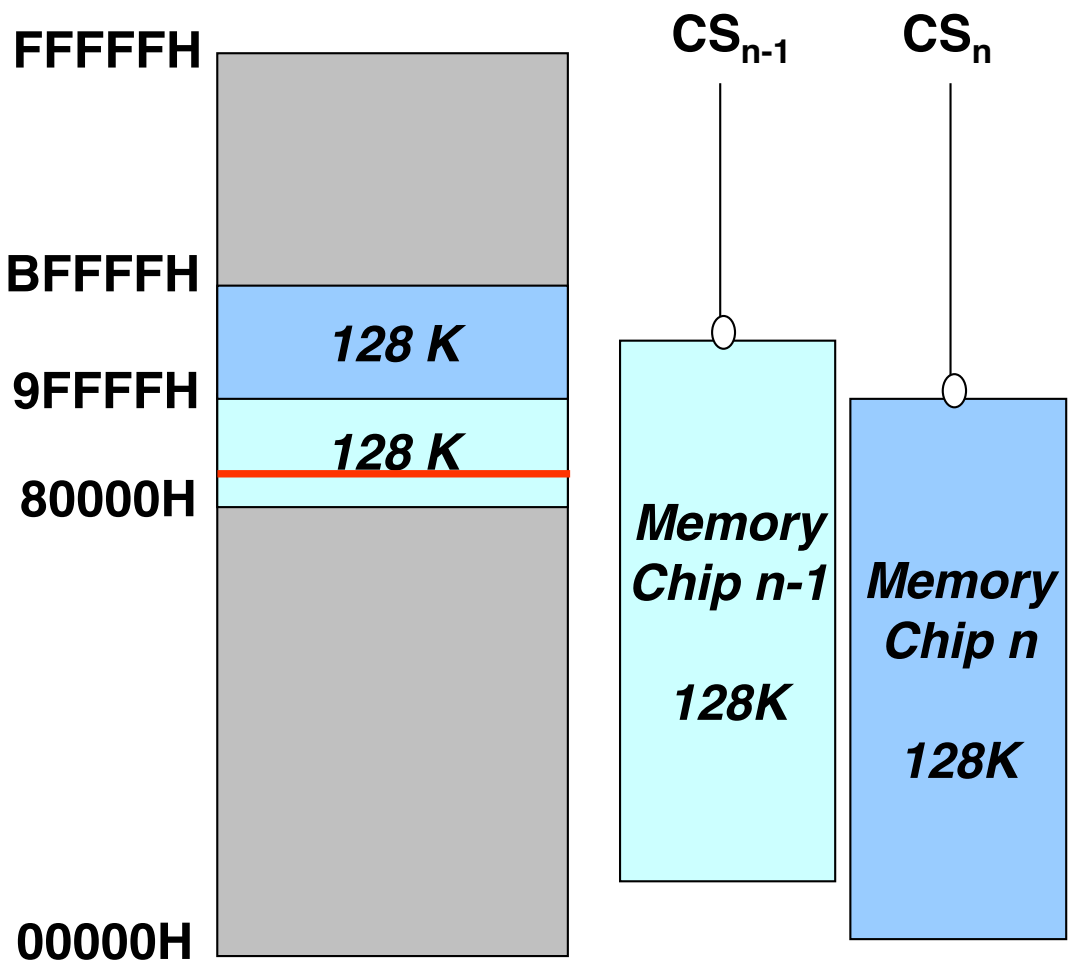
1. Suppose the following **memory address map**:

   Chip N-1: 80000H-9FFFFH
   Chip N: A0000H-BFFFFH

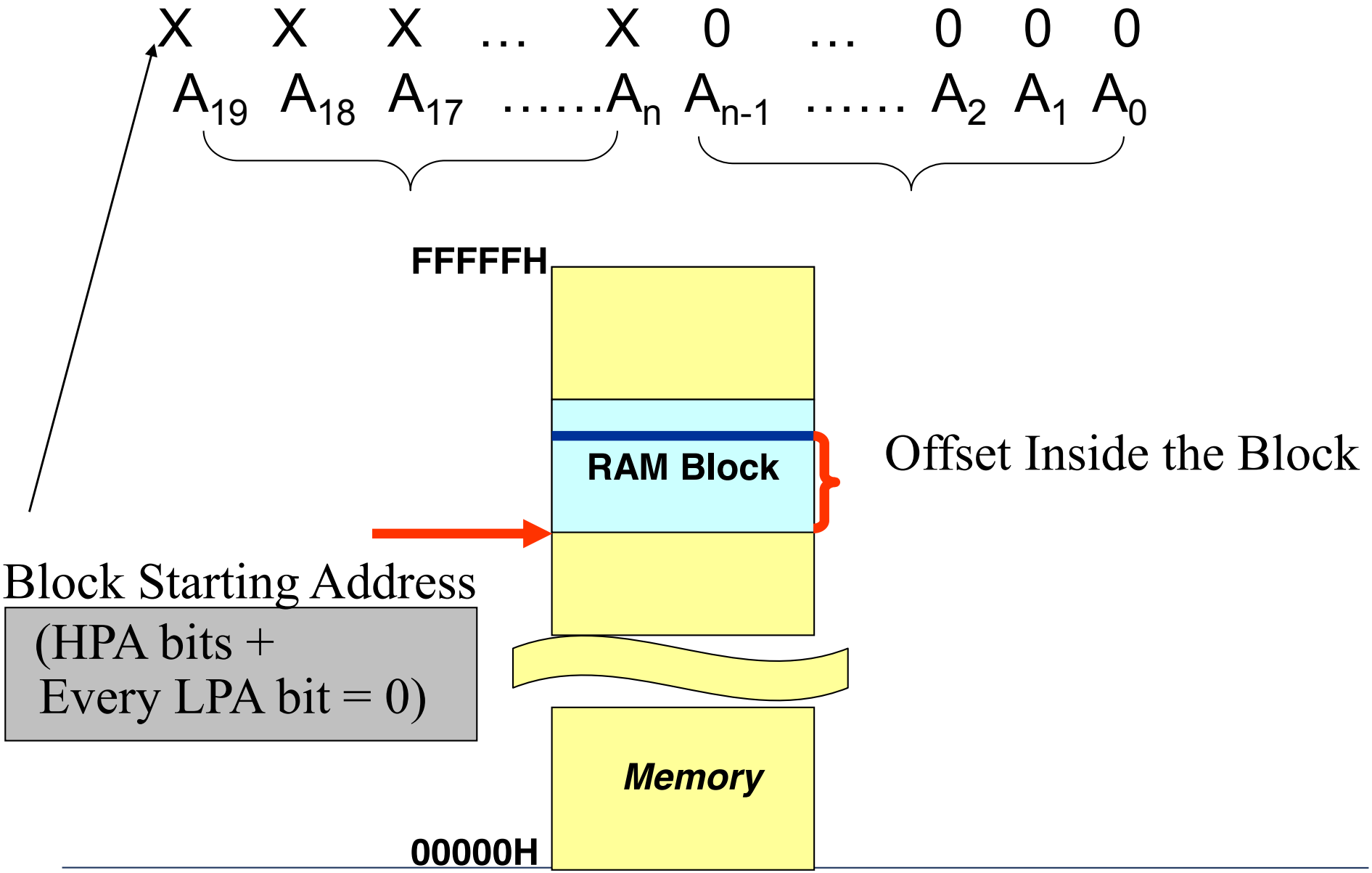
   If you want to read the data at 80028H, which memory chip should be activated?

2. The **logic function of chip select signal CS_n** for each memory chip answers this type of question.

   $CS_{n-1} = A_{19} \backslash A_{18} \backslash A_{17}$

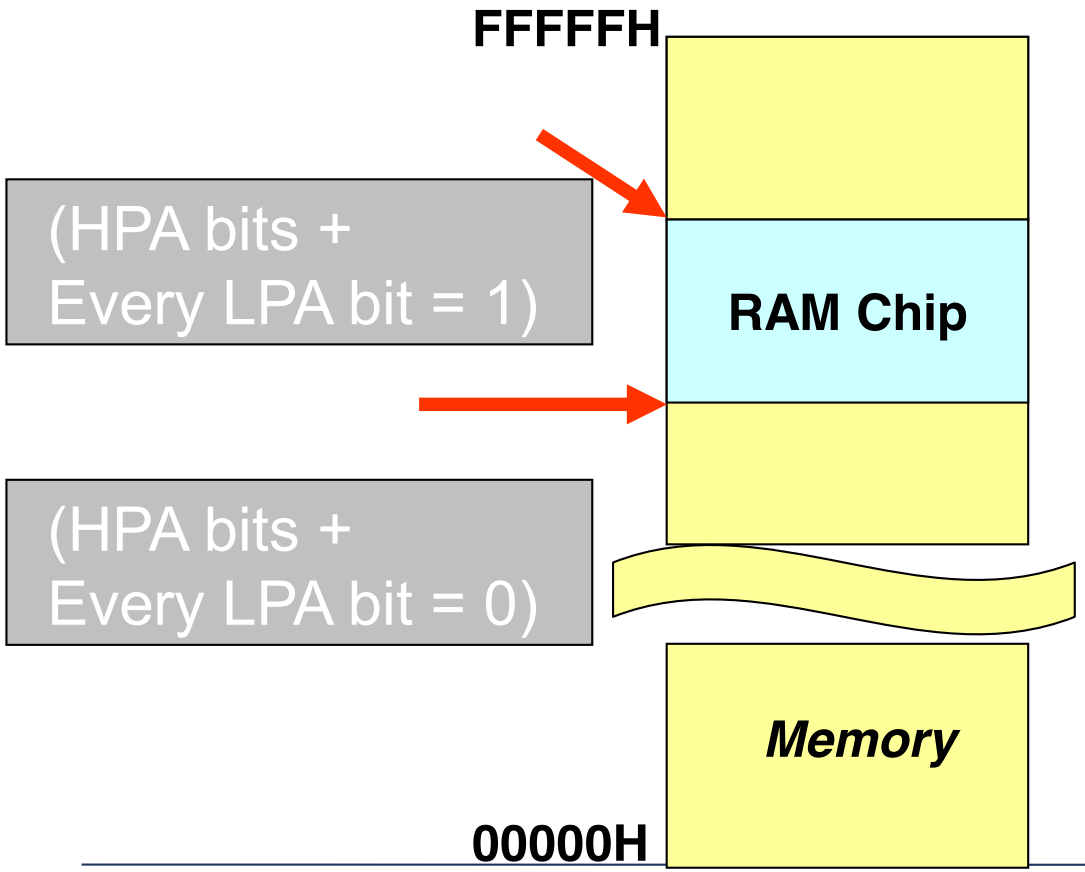


A Two-Portion Memory Address



Chip Select Signal Logic Function

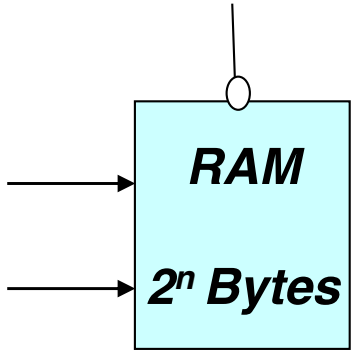
A_{19} A_{18} A_{17} ……A_n A_{n-1} …… A_2 A_1 A_0



When an RAM chip is mapped to an address range, its CS\ logic function is:

$\overline{CS} = f(HPA)$

$\overline{CS} = f(A_{19}\text{-}A_n)$

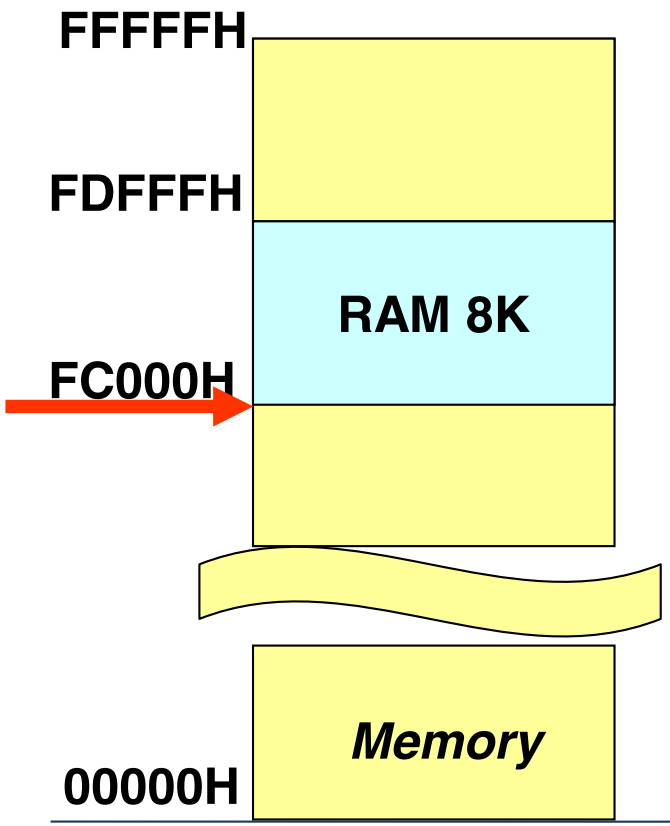


Chip Select Signal Logic Function Example

												1	1	1	1	1	1	1	1	1	1	1	1	1
												0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	0	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
A_{19}	A_{18}	A_{17}	A_{16}	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0					

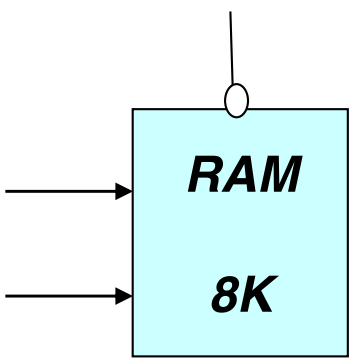
Block Starting Address

Block Internal Offset



When the RAM chip is mapped to the address range FC000H-FDFFFH, its CS\ logic function is:

$$\overline{CS} = \overline{A_{19} A_{18} A_{17} A_{16} A_{15} A_{14} A_{13}}$$



Processor & Accelerator based Computing Architectures

- I/O interfacing
- Addressing
- Interrupt
- Direct memory access

Introduction to Interrupts

- A computer system normally connects to I/O and other devices
  - keyboard
  - monitor
  - mouse
  - sensors etc.
- Processor needs to serve these devices at **unpredictable times** (e.g. CPU does not know when you are about to hit a key)

Solutions

- Alternative 1: Polling
  - Every so often, processor checks each device to see if it has a request
    - Takes CPU time even if no request is pending
    - How does software know when to let the system poll?
- Alternative 2: Interrupts
  - Give each device a wire (interrupt line) that it can use to signal the processor
    - When an interrupt happens, a processor executes a routine called an interrupt service routine (ISR) to deal with the interrupt
    - No overhead when no requests are pending

Polling vs Interrupt

- Imagine processor is like a director working in his office, and devices are guests

  – Polling is like the director who goes to open the door every period of time to see if a guest is there.

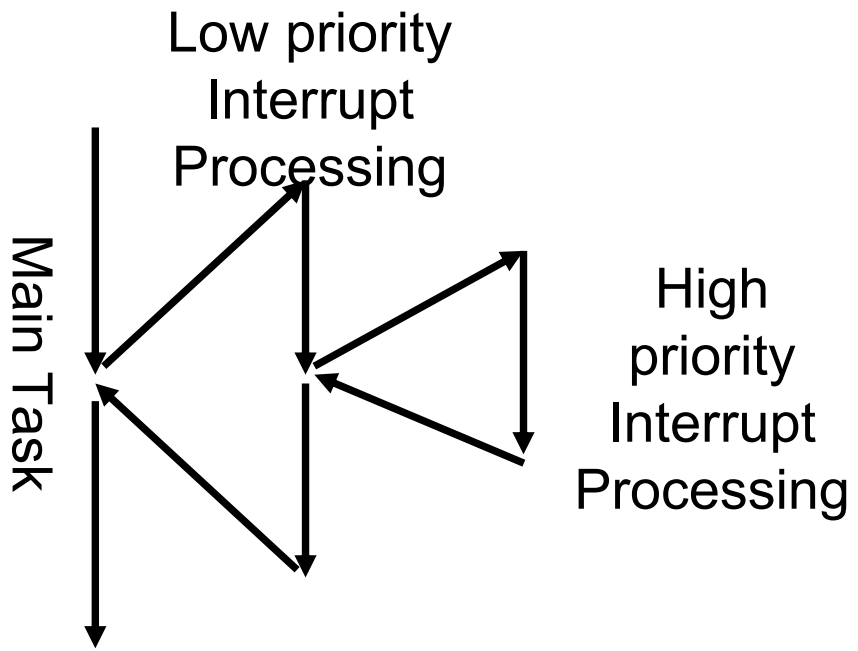
  – Interrupt is like when a guest comes, he knocks on the door. Director then stops what he is doing and prepares to welcome the guest.

Polling vs Interrupt

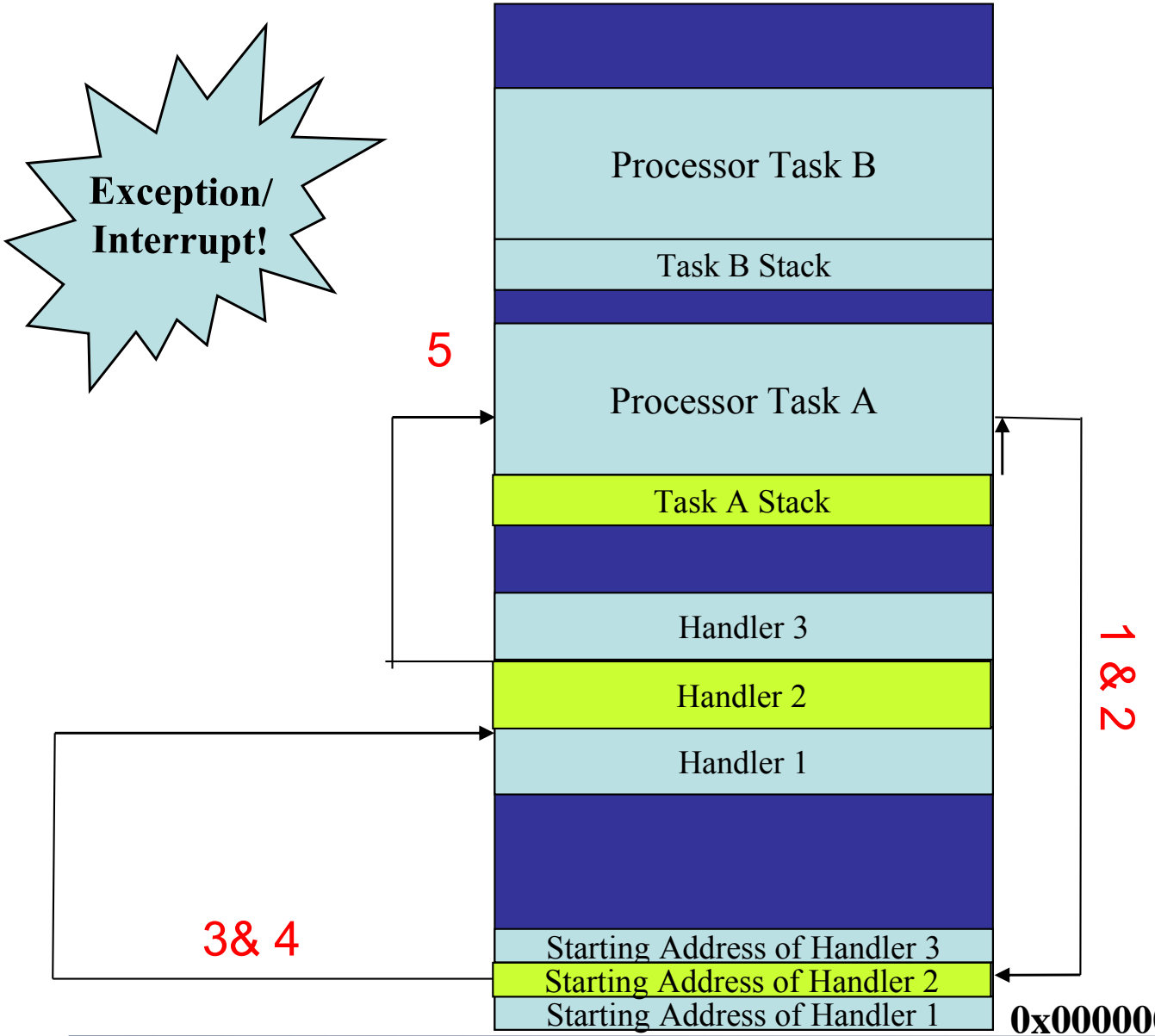
- Polling
  - Advantages:
    - CPU determines how often it needs to poll
    - Response time is faster (if the device is being polled regularly)
  - Disadvantages:
    - CPU gets into busy wait
      - —*CPU needs to check the device many times*
      - —*Most I/O devices operate at a much slower speed than CPU*
      - —*Accessing I/O devices can take a long time, e.g. disk reads/writes ~10ms latency*
- Interrupt
  - Advantages:
    - More efficient since CPU can do other work
  - Disadvantages:
    - Response time is slower*
      - —*CPU has to stop working, determine which device is requesting service, save its work before serving the request*
    - *compared to the above case of regularly polling a particular device. However, if the polling loop includes many devices which may require service, interrupt response time is likely to be faster than polling.

Interrupt Processing

1. CPU is executing the main task
2. A device/peripheral requests for service using an interrupt
3. Based on the interrupt configuration and priority, CPU decides if it allows that interrupt
4. If yes, save all the status onto stack, run the interrupt handler to serve the I/O device
5. When finish, CPU reloads all
6. status before processing interrupt,
7. and continues the main task
8. A higher priority interrupt can
9. interrupt or pre-empt the lower priority one



Interrupt Processing Sequence

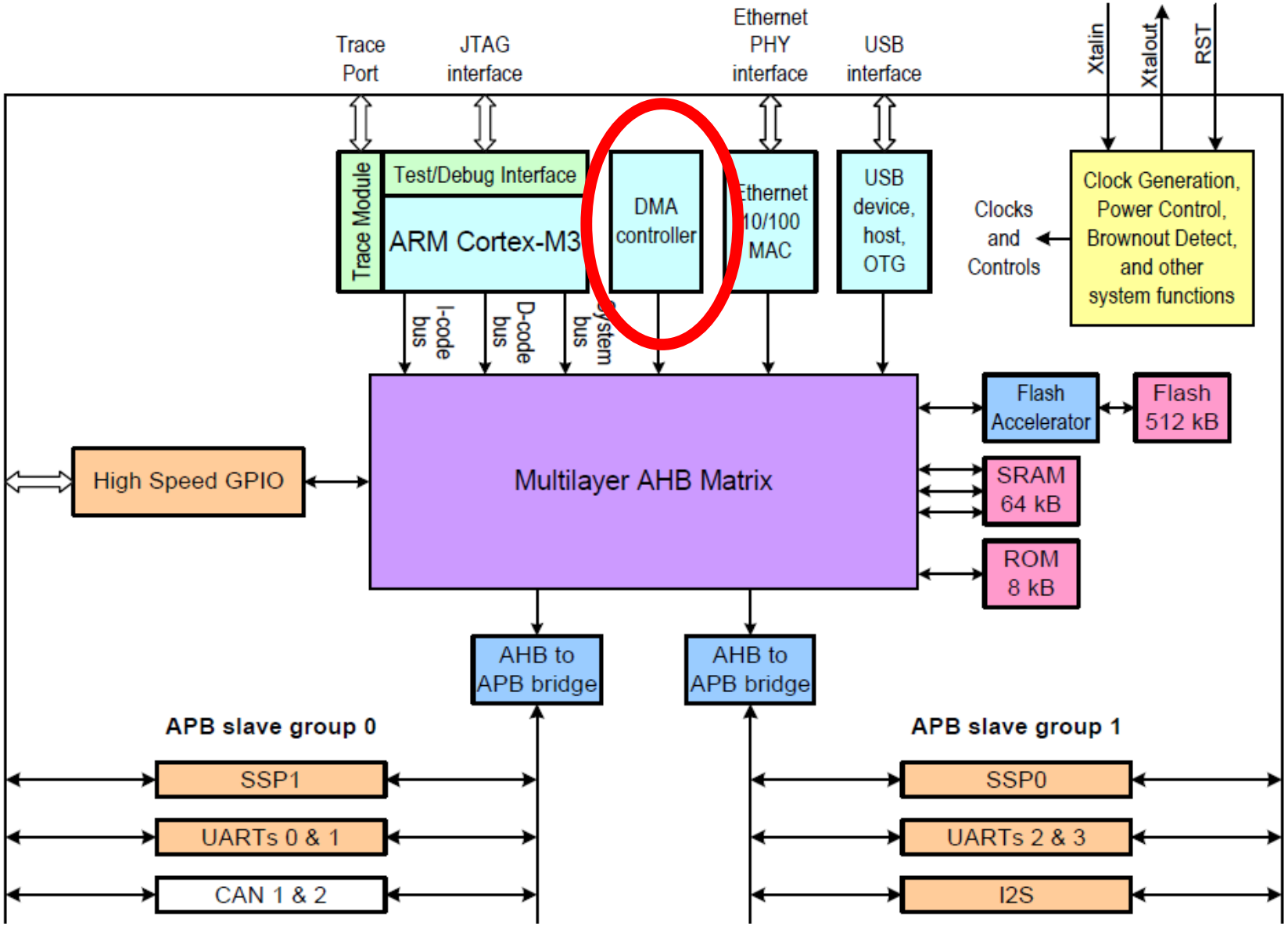


1. μP finishes current instruction in Task A.
2. μP saves current context on stack.
3. μP gets an handler (Exception/Interrupt handler) address from Exception/Interrupt Vector Table
4. μP jumps to handler and begins executing it.
5. μP finishes handler execution, restores previous context from stack and returns to Task A.

Processor & Accelerator based Computing Architectures

- I/O interfacing
- Addressing
- Interrupt
- ☛ Direct memory access

NXP LPC17xx Simplified Block Diagram



Transfer Bulk Data Efficiently with DMA

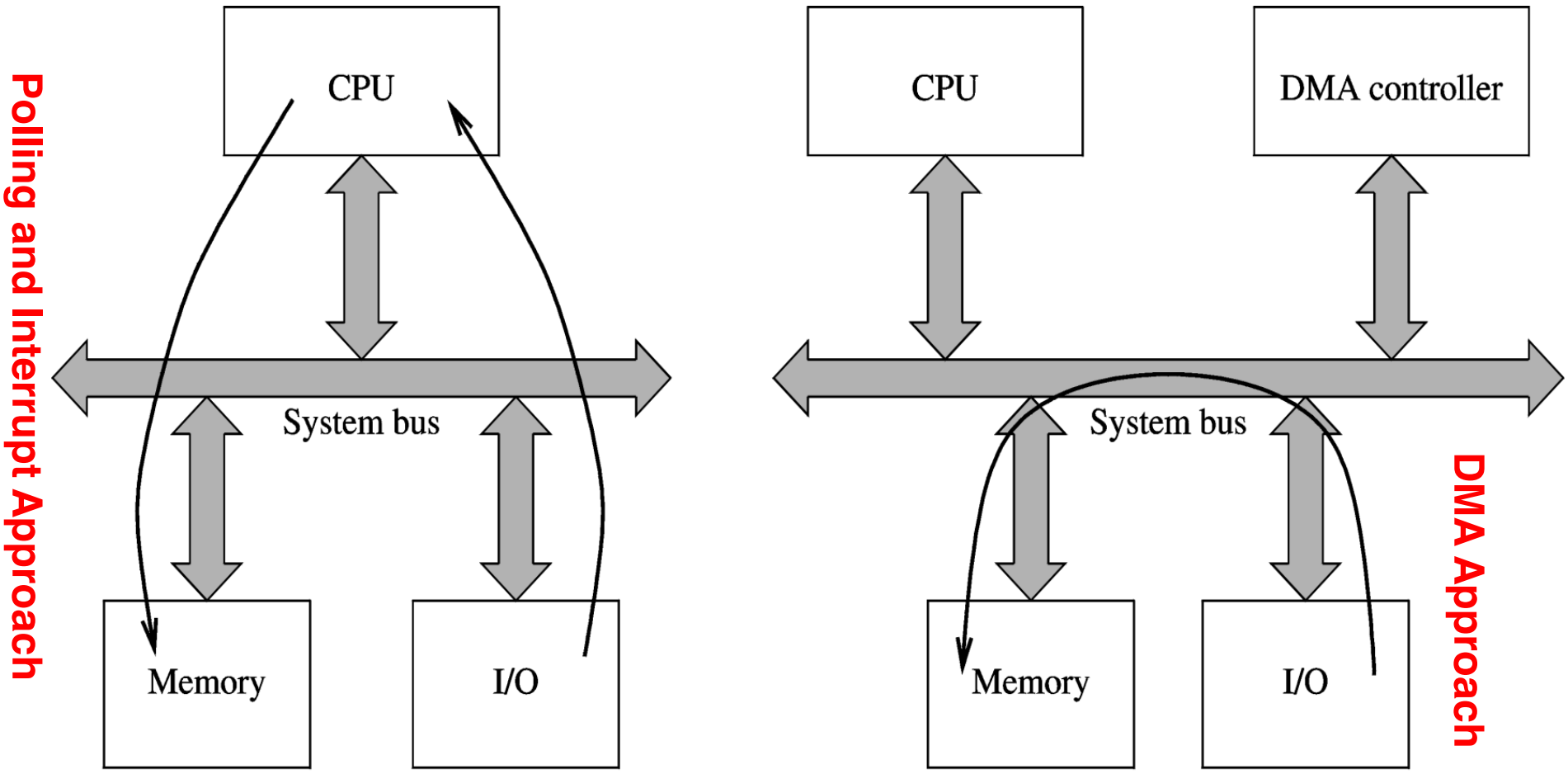
From time to time, we need to transfer bulk data between memory and I/O devices, such as floppy disk or CRT display.

- **It wastes a lot of time for the CPU just transfer data from source to destination using polling or interrupt driven data transfer.**

- **The alternative way of transferring bulk data is the direct memory access (DMA) technique, in which the data is transferred directly between memory and I/O devices under the control of a DMA controller**

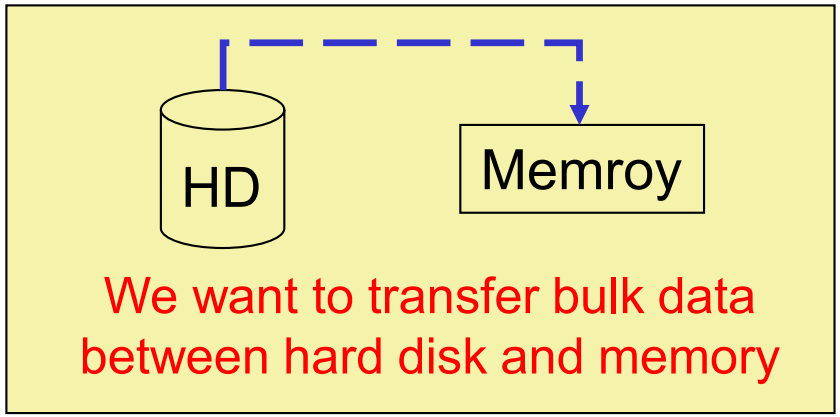
- **A DMA controller is designed to complete the bulk data transfer much faster than the CPU.**

DMA: Direct Transfer Between I/O and Memory

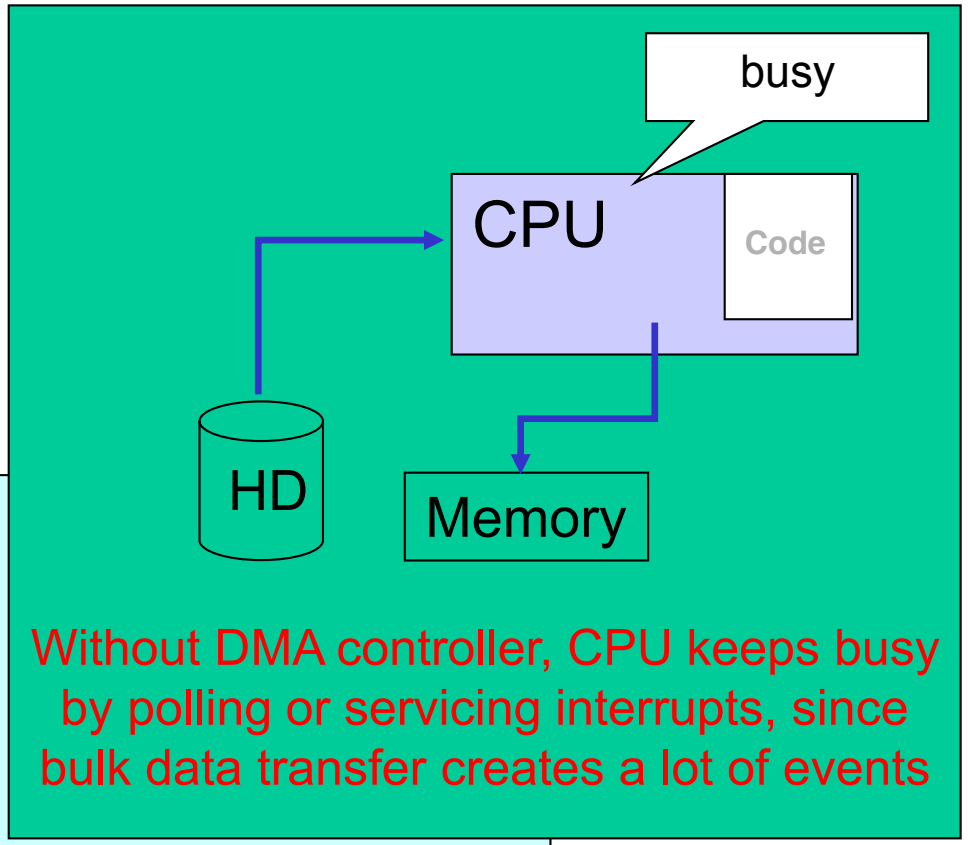


Polling and interrupt based data transfers need CPU to take part in, while DMA transfer does not!

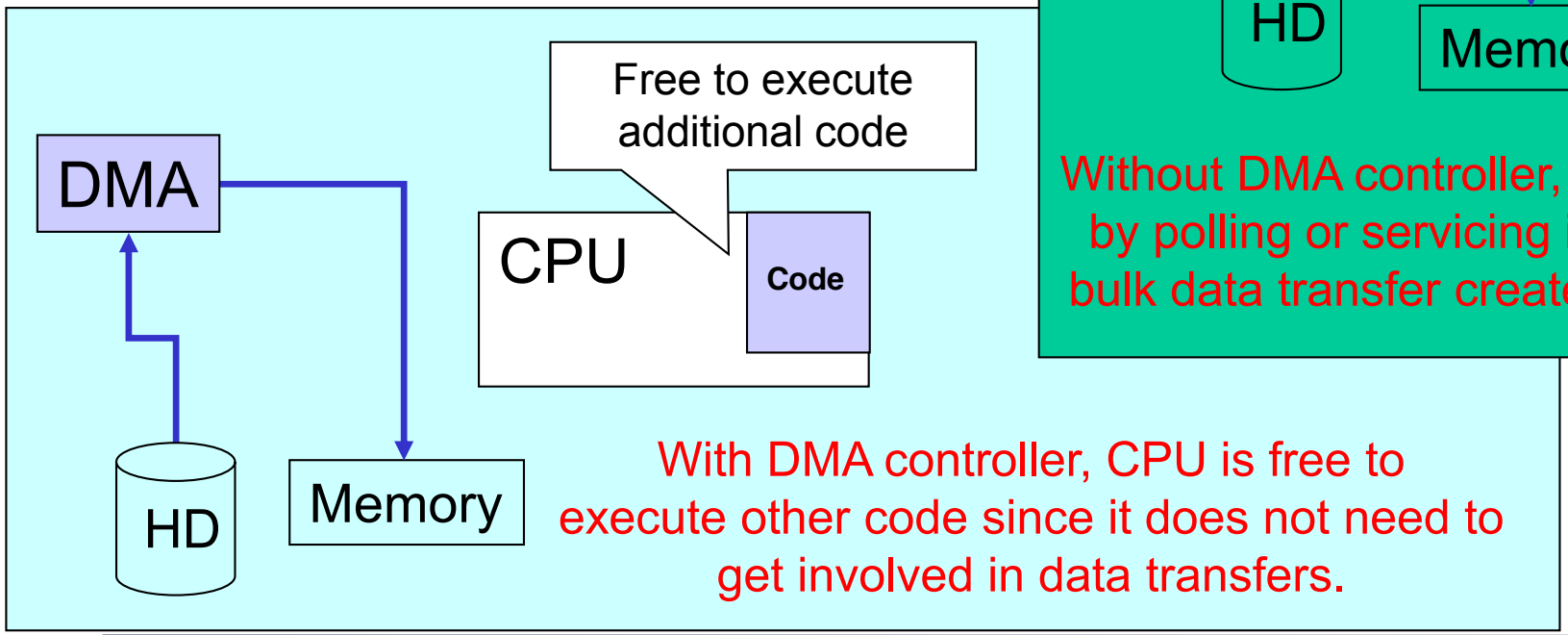
DMA Transfer Frees CPU



We want to transfer bulk data between hard disk and memory



Without DMA controller, CPU keeps busy by polling or servicing interrupts, since bulk data transfer creates a lot of events



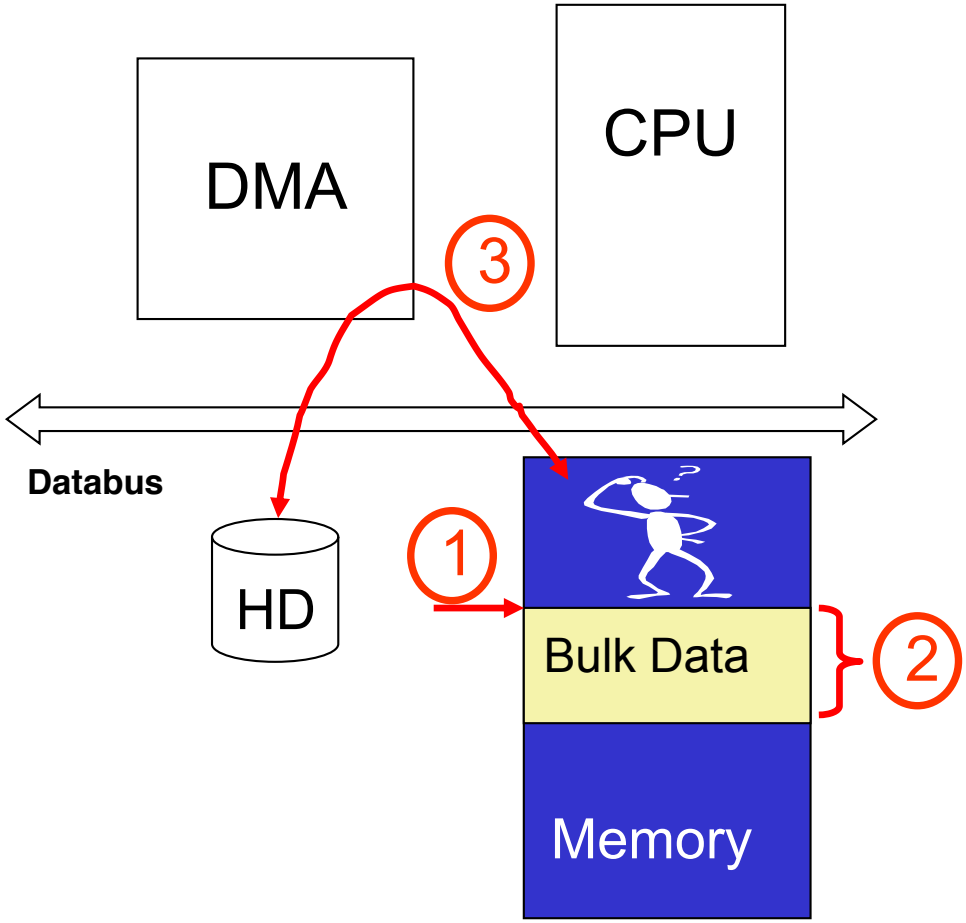
With DMA controller, CPU is free to execute other code since it does not need to get involved in data transfers.

Comparison of the Three Ways of Data Transfer

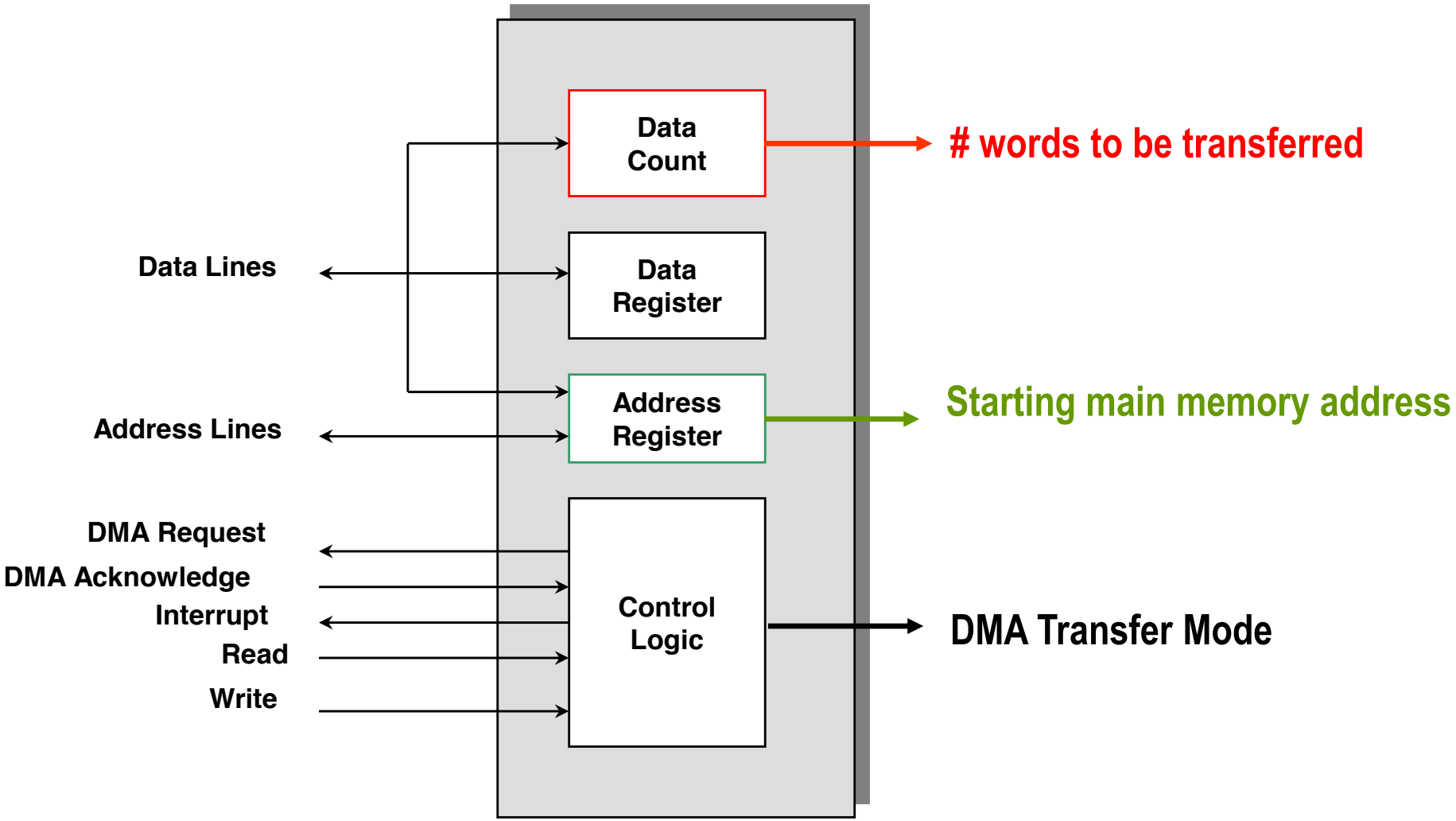
- **Polling Approach**
  - Processor polls I/O device for data
  - Keeps processor busy and wastes resources
- **Interrupt Approach**
  - I/O interrupts processor
  - Processor transfers data from/to memory
  - Processor is only busy when there is interrupt
  - Not good when interrupts are quite frequent in bulk data transfer
- **DMA Approach**
  - DMA Controller transfers data from/to memory
  - Controller only interrupts processor when transfer finishes
  - Good in bulk data transfer

DMA Controller

- **DMA transfer is implemented using a DMA controller**
  - DMA controller
    - **Acts as slave to processor**
    - **Receives instructions from processor**
    - **Example: Reading from an I/O device**
  - Processor gives details to the DMA controller
    1. **Main memory starting address**
    2. **Number of bytes to transfer**
    3. **Transfer mode (memory → I/O device, or vice versa)**



Typical DMA Controller Block Diagram



Summary

- Processor uses address bus, data bus and control bus to communicate with memory and I/O devices.
- All memory devices and I/O devices are mapped to a address space. Processor uses the address map to know where is the target memory or I/O device.
- Interrupt frees CPU from always monitoring I/O devices.
- DMA frees CPU from moving data between memory and I/O devices.